

THE UNIVERSITY OF WESTERN AUSTRALIA

Lecture 2
Variables and numeric data types

Objectives of this Lecture

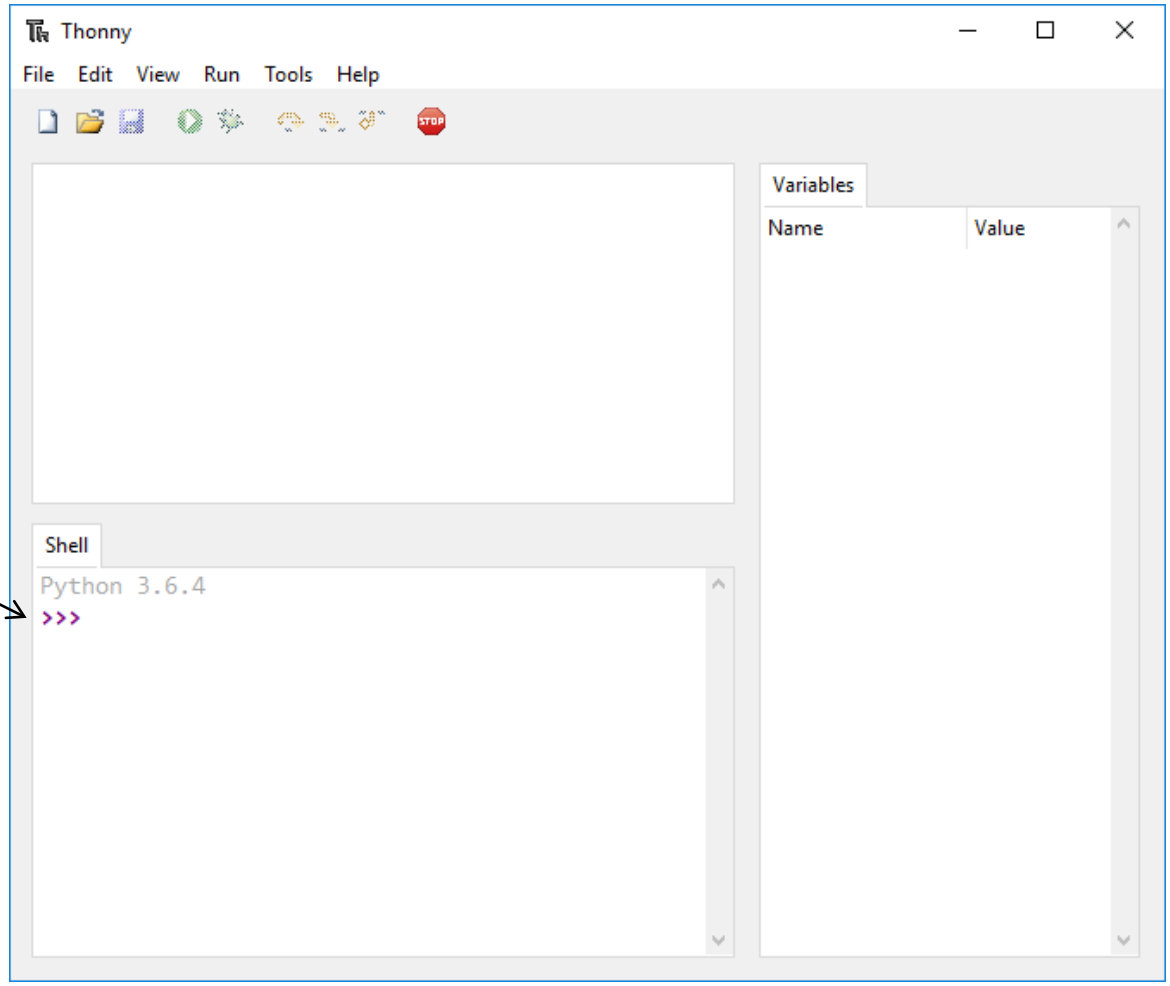
- Getting started with Python/Thonny
- To understand the process of assigning values to variables
- To understand simultaneous or unary operator assignment
- To look into limitations of numeric data types

Getting stated with Thonny

Thonny is an IDE (Integrated Development Environment)

Simple IDE for beginners to manage, execute and debug Python codes

The **>>>** that you see is a prompt that indicates the interpreter is waiting for you to type a Python statement.



https://thonny.org/

Getting Started with Python

Start with single statements

```
>>> 2+3
5
>>> 22/7
3.142857142857143
>>> 3**2
9
>>> print("Hello world")
Hello world
>>> print("2+3=", 2+3)
2+3=5
```

slido

Please download and install the Slido app on all computers you use





What is a variable in Python?

ⓘ Start presenting to display the poll results on this slide.

Variable

- What is a variable?

A variable is a name/identifier that represents a value in the computer's memory.

- Every variable is connected to a value
- When a variable represents a value in the computer's memory, we say that the variable references the value.



Creating Variables with Assignment Statements

- You use an assignment statement to create a variable and make it reference a piece of data.

```
age = 25
```

After this statement executes, a variable named age will be created, and it will reference the value 25.



Assignment Statements

- Simple Assignment statement

variable = expression

The equal sign (=) is known as the assignment operator.

- expression is a value, or any piece of code that results in a value. After an assignment statement executes, the variable listed on the left side of the = operator will reference the value given on the right side of the = operator.

Programs are composed of statements that are built from identifiers and expressions

Assignment Statements – the Simple View

An expression can be composed of:

- **Variable**: an identifier that references a value.
- **Literal**: is a representation of a specific value. For example, 1 is a literal.
- **Operator**(s): enable operations to be performed on literals and variables

```
x = x+1
```

Examples of python operators for numbers include:

+ (addition), - (subtraction), * (multiplication), / (division), % (modulo), ** (exponentiation)

Assignment Statements

- `x = 3.9 * x * (1-x)`
- `fahrenheit = 9/5 * celsius + 32`
- `x = 5`

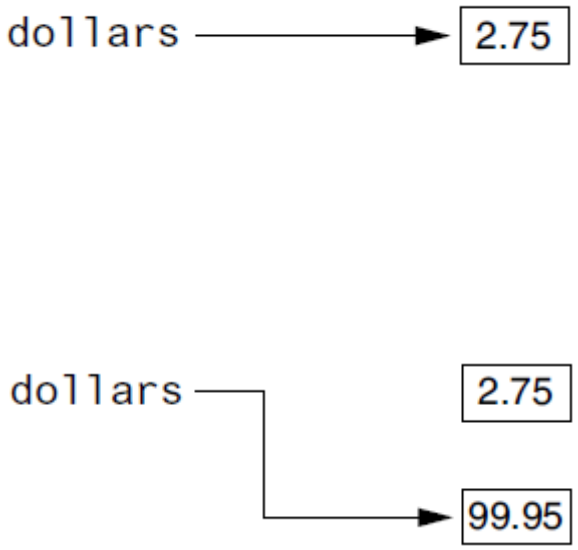
The spacing around parts of assignments is optional but makes the resulting program much more readable – and hence maintainable.

Assignment Statements

Variables can be reassigned as many times as you want!

```
>>> dollars = 2.75
>>> dollars
2.75

>>> dollars = 99.95
>>> dollars
99.95
```



Simultaneous Assignment

- Several values can be calculated at the same time
- <var>, <var>, … = <expr>, <expr>, …
- Evaluate the expressions on the right and assign them to the variables on the left
  - *Must have same number of expressions as variables!*

```
>>>sum, diff = x+y, x-y
```

Simultaneous Assignment

- How could you swap the values for x and y?
- Let x= 10 and y =15
  - *Does this work?*

```
x = y
y = x
```

slido

Please download and install the Slido app on all computers you use



What is the new value of y?

ⓘ Start presenting to display the poll results on this slide.

Simultaneous Assignment

- How could you swap the values for x and y?
  - *Why doesn't this work?*

```
x = y
y = x
```

  - *#assume x= 10, y= 15 initially….. x= y will make x= 15, and then y=x, will assign y the value of x*

- We could use a temporary variable…

```
temp = x
x = y
y = temp
```

Simultaneous Assignment

We can swap the values of two variables easily in Python!

```
>>> x = 3
>>> y = 4
>>> print(x, y)
3 4
>>> x, y = y, x
>>> print(x, y)
4 3
```

Unary operator Assignment

- Certain types of assignment statement are so common that a short-cut exists

  `x = x + n,` (especially `x = x +1`)

  `x = x - n` (n can be any expression)

- These become

  `x += n`

  `x -= n`

- `Also`

  `x *= n`

  `x /= n`, *etc.*

Numeric Data Types

- There are two different kinds of numbers!
  - *5, 4, 3, 6 are whole numbers – they don't have a fractional part*
  - *0.25, 1.10, 3.142 are decimal fractions*

- Inside the computer, whole numbers and decimal fractions are represented quite differently!
  - *We say that decimal fractions and whole numbers are two different data types.*

Numeric Data Types

Integers

- Whole numbers are represented using the *integer* (*int*) data type.
  - *Size depends on machine using it*

- These values can be positive or negative whole numbers.

Numeric Data Types

Floats:

Numbers with a decimal point are called floats or *floating-point* values.

- How can we tell which is which?
  - *A numeric literal without a decimal point produces an int value*
  - *A literal that has a decimal point is represented by a float (even if the fractional part is 0)*

Numeric Data Types

- Why do we need two number types?
  - *Values that represent counts can't be fractional*
  - *Most mathematical algorithms are very efficient with integers*
  - *The float type stores only an approximation to the real number being represented!*
  - *Since floats aren't exact, use an int whenever possible!*

Numeric Data Types

- Operations on ints produce ints (excluding /)
- Operations on floats produce floats.

```
>>> 3.0+4.0
7.0
>>> 3+4
7
>>> 10.0/3.0
3.3333333333333335
>>> 10/3
3.3333333333333335
>>> 10 // 3
3
```

/ does floating point division

// does integer division

If you mix an integer and a float in any other operation, you'll get a float

Numeric Data Types

- Integer division produces a whole number.
  - *That's why* `10//3` gives `3`

- Think of it as 'goes into', where 10//3 gives 3 since 3 (goes into) 10, three times (with a remainder of 1)

- 10 % 3 = 1 is the remainder of the integer division of 10 by 3.

Limits of Int

- What's going on?
  - *While there are an infinite number of integers, there is a finite range of integers that can be represented by* int.
  - *This range depends on the number of bits a particular CPU uses to represent an integer value.*
- Does switching to *float* data types get us around the limitations of *int*?

Approximation of float

```
>>> x1 = 10**121
10000…
>>> y1 = x1 + 1
10000… … … 1
>>> z1 = x1 - y1
-1
>>> x2 = 10.0**121 # can also be x2 = 10e+121
1e+122
>>> y2 = x2 + 1
1e+122
>>> z2 = x2 - y2
0.0
```

Handling Large Integers

- Floats are approximations
- Floats allow us to represent a larger range of values, but with fixed precision.
- Python *int* is not a fixed size but expands to handle whatever value it holds.
- Newer versions of Python automatically convert *int* to an expanded form when it grows so large as to overflow.
- Can store and work with indefinitely large values (e.g., 100!) at the cost of speed and memory

Type Conversion

- Combining an `int` with a `float` in an expression will return a `float`
- We can explicitly convert between different data types
- For example, the `int` and `round` functions convert a float to integer

```
>>> int(6.8)    # Truncate
6
>>> round(6.8)
7

>>> float(6)
6.0
```

Type Conversion : More examples

Conversion function	Example use	Value returned
int(<a number or string>)	int(2.87)	2
int	int(“55”)	55
float(<a number or string>)	float(32)	32.0
str(<any value>)	str(43)	‘43’

str: string/text

The int() and float() functions work only if the item that is being converted contains a valid numeric value. If the argument cannot be converted to the specified data type, an error occurs.

Find Data Type

- We can use the type function to find the data type

```
>>> type(4)
<class 'int'>

>>> type(4.3)
<class 'float'>

>>> x = 5.76
>>> type(x)
<class 'float'>

>>> type(hello) # Without quotes, assumes 'hello'  is a
                # variable. Not defined, so will generate
                # an error
>>> type('hello')
<class 'str'>
```

If a variable exists, the type of the variable is the type of the value assigned to it

Scientific Notation

Decimal Notation	Scientific Notation	Meaning
2.75	2.75e0	2.75×10^{0}
27.5	2.75e1	2.75×10^{1}
2750.0	2.75e3	2.75×10^{3}
0.0275	2.75e-2	2.75×10^{-2}

- Typical precision is 16 digits (decimal places)

Float Problems

- Very large and very small floating-point values can also cause problems (current Python)

```
>>> x = 1.0e308
>>> x
1e+308
>>> x = 100* x
>>> x
inf
```

This is called over-flow

```
>>> x = 1.0e-323
>>> x
1e-323
>>> x = x / 100.0
>>> x
0.0
```

This is called under-flow

Lecture Summary

- Understanding the concept of assignment in Python

- We learned how to
  - *assign values to variables*
  - *do multiple assignments in one statement*
  - *Limitations of data types*

slido

Please download and install the Slido app on all computers you use



You cannot use a variable until you have assigned a value to it. An error will occur if you try to perform an operation on a variable, such as printing it, before it has been assigned a value.

ⓘ Start presenting to display the poll results on this slide.